

**LAPORAN PRAKTIKUM 7**

**SORTING ALGORITHM**

**STRUKTUR DATA**



Disusun Oleh:

Nama: Amiratul Fadhilah

NIM: 2511532023

Dosen Pengampu: Dr. Ir. Wahyudi, S. T., M. T

**DEPARTEMEN INFORMATIKA**

**FAKULTAS TEKNOLOGI INFORMASI**

**UNIVERSITAS ANDALAS**

**PADANG**

**2026**

## KATA PENGANTAR

Puji syukur ke hadirat Allah Yang Maha Esa atas rahmat dan karunia-Nya, sehingga penulisan laporan praktikum dengan judul “Sorting Algorithm” ini dapat diselesaikan tepat waktu. Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas mata kuliah Praktikum Struktur Data.

Laporan ini akan membahas secara mendalam mengenai konsep, logika dasar, serta implementasi berbagai algoritma pengurutan data (Sorting) menggunakan bahasa pemrograman Java. Fokus utama pembahasan mencakup tiga algoritma yaitu *Insertion Sort*, *Selection Sort*, dan *Bubble Sort*, serta implementasi visualisasinya menggunakan GUI Java Swing. Diharapkan melalui pembahasan ini, pemahaman mengenai bagaimana data dikelola, diurutkan, dan divisualisasikan secara efisien dalam memori komputer dapat tergambar dengan jelas.

Penulis menyadari bahwa masih terdapat berbagai kekurangan, baik dari segi penulisan maupun analisis kode program. Oleh karena itu, kritik dan saran yang membangun dari dosen pengampu maupun asisten praktikum sangat diharapkan demi perbaikan di masa mendatang. Semoga dokumentasi praktikum ini dapat memberikan manfaat tambahan bagi pembaca.

Padang, 19 Mei 2026

Amiratul Fadhillah

## DAFTAR ISI

|                                              |    |
|----------------------------------------------|----|
| KATA PENGANTAR.....                          | i  |
| DAFTAR ISI.....                              | ii |
| BAB I PENDAHULUAN                            |    |
| 1.1 Latar Belakang.....                      | 1  |
| 1.2 Tujuan Praktikum .....                   | 1  |
| 1.3 Manfaat Praktikum .....                  | 2  |
| BAB II PEMBAHASAN                            |    |
| 2.1 Landasan Teori .....                     | 3  |
| 2.2 Alat dan Bahan.....                      | 3  |
| 2.3 Langkah Praktikum dan Kode Program ..... | 4  |
| 2.3.1 Kelas Insertion Sort.....              | 4  |
| 2.3.2 Kelas Selection Sort.....              | 5  |
| 2.3.3 Kelas Bubble Sort.....                 | 6  |
| 2.3.4 Kelas Insertion Sort GUI .....         | 7  |
| BAB III PENUTUP                              |    |
| 3.1 Kesimpulan.....                          | 13 |
| 3.2 Saran.....                               | 13 |
| DAFTAR PUSTAKA.....                          | 14 |
| TUGAS 7 PRAKTIKUM STRUKTUR DATA.....         | 15 |

# BAB I

## PENDAHULUAN

### 1.1 Latar Belakang

Dalam dunia ilmu komputer dan rekayasa perangkat lunak, efisiensi pengelolaan data adalah salah satu aspek krusial yang menentukan kualitas sebuah program. Seiring dengan bertambah besarnya volume informasi yang diproses, data yang acak dan tidak terstruktur akan memperlambat kinerja sistem, terutama saat melakukan operasi pencarian. Oleh karena itu, diperlukan teknik khusus untuk menyusun data ke dalam urutan tertentu, baik secara ascending maupun descending yang dikenal dengan istilah pengurutan atau *Sorting*.

Terdapat berbagai macam algoritma pengurutan yang telah dikembangkan, masing-masing dengan karakteristik, kelebihan, dan kelemahan tersendiri bergantung pada kondisi data yang ditangani. Algoritma dasar seperti *Insertion Sort*, *Selection Sort*, dan *Bubble Sort* sering kali menjadi fondasi awal bagi mahasiswa untuk memahami logika komputasi dan manipulasi elemen di dalam struktur data array. Meskipun algoritma-algoritma ini mungkin tidak secepat algoritma tingkat lanjut seperti *Quick Sort* pada jumlah data masif, ketiganya sangat unggul dalam kesederhanaan logika dan kemudahan implementasi dalam bahasa pemrograman Java.

Lebih lanjut, pemahaman terhadap algoritma tidak hanya terbatas pada eksekusi teks di konsol. Visualisasi langkah demi langkah menggunakan antarmuka grafis (GUI) sangat membantu dalam membedah proses perpindahan memori secara interaktif. Melalui pemanfaatan *library* Swing pada Java, proses *sorting* yang abstrak dapat diobservasi secara langsung, memperkuat fondasi logika algoritmik sekaligus melatih keterampilan perancangan antarmuka visual.

### 1.2 Tujuan Praktikum

Adapun tujuan dari pelaksanaan praktikum struktur data mengenai Sorting Algorithm ini adalah sebagai berikut:

- 1) Memberikan pemahaman teoretis dan praktis mengenai konsep pengurutan data menggunakan algoritma Insertion Sort, Selection Sort, dan Bubble Sort.
- 2) Melatih kemampuan merancang, menulis, dan menganalisis kode program Java untuk manipulasi array satu dimensi.
- 3) Membekali keterampilan merancang antarmuka visual (GUI) interaktif menggunakan Java Swing.
- 4) Mengasah kemampuan membandingkan efisiensi dan cara kerja spesifik dari berbagai algoritma dasar.

### **1.3 Manfaat Praktikum**

Praktikum ini diharapkan memberikan manfaat yang signifikan secara akademis maupun teknis. Secara akademis, konsep abstrak mengenai manajemen memori dan kompleksitas waktu yang dipelajari di kelas dapat dipahami wujud nyatanya melalui uji coba kode. Secara teknis, kebiasaan menulis sintaks Java yang terstruktur akan meningkatkan ketelitian dalam mencegah `IndexOutOfBoundsException`, serta memperkaya skill set mahasiswa dalam mengembangkan aplikasi berbasis antarmuka grafis (GUI).

## BAB II

### PEMBAHASAN

#### 2.1 Landasan Teori

Algoritma *Sorting* adalah serangkaian langkah-langkah logis yang dirancang untuk mengatur sekumpulan data ke dalam urutan sistematis, data yang diurutkan dapat berupa angka, huruf, nama siswa, nilai barang, dan lain-lain. Tiga algoritma dasar yang dibahas pada praktikum kali ini memiliki cara kerja yang berbeda dalam memindahkan data:

- Insertion Sort: Algoritma ini bekerja dengan mengambil satu elemen data dan menyisipkannya pada posisi yang tepat di antara elemen-elemen sebelumnya yang sudah terurut.
- Selection Sort: Membagi array menjadi dua bagian: bagian yang sudah terurut dan yang belum. Algoritma ini mencari nilai minimum (atau maksimum) dari bagian yang belum terurut, lalu menukarnya dengan elemen paling depan dari bagian tersebut.
- Bubble Sort: Bekerja dengan cara membandingkan elemen yang berdekatan secara berpasangan dan menukarnya jika posisinya salah. Proses ini diulang hingga seluruh deret terurut

Berikut adalah tabel perbandingan kompleksitas waktu rata-rata dari ketiga algoritma tersebut:

| Nama Algoritma | Best Case | Average Case | Worst Case | Sifat Stabilitas |
|----------------|-----------|--------------|------------|------------------|
| Insertion Sort | $O(n)$    | $O(n^2)$     | $O(n^2)$   | Stabil           |
| Selection Sort | $O(n^2)$  | $O(n^2)$     | $O(n^2)$   | Tidak Stabil     |
| Bubble Sort    | $O(n)$    | $O(n^2)$     | $O(n^2)$   | Stabil           |

#### 2.2 Alat dan Bahan

- 1) Perangkat keras (*Hardware*): Komputer atau Laptop.
- 2) Sistem Operasi (OS): Windows, macOS, atau distribusi Linux.

- 3) Perangkat Lunak (*Software*): Java Development Kit (JDK), serta Integrated Development Environment (IDE) seperti Eclipse.

## 2.3 Langkah Praktikum

### 2.3.1 Kelas Insertion Sort

Kode program di bawah ini mendemonstrasikan implementasi algoritma Selection Sort pada bahasa Java.

```
package pekan7_2511532023;

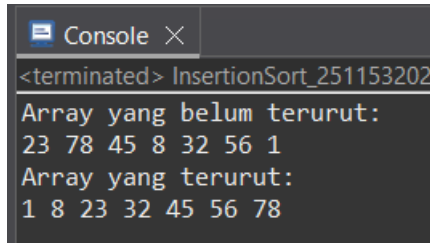
public class InsertionSort_2511532023 {
    public static void insertionSort_2023(int[] arr_2023) {
        int n_2023 = arr_2023.length;
        for (int i_2023 = 1; i_2023 < n_2023; i_2023++) {
            int key_2023 = arr_2023[i_2023];
            int j_2023 = i_2023 - 1;
            while (j_2023 >= 0 && arr_2023[j_2023] > key_2023) {
                arr_2023[j_2023 + 1] = arr_2023[j_2023];
                j_2023--;
            }
            arr_2023[j_2023 + 1] = key_2023;
        }
    }
    public static void main(String[] args) {
        int arr_2023[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_2023 = arr_2023.length;
        System.out.printf("Array yang belum terurut:\n");
        for (int i_2023 = 0; i_2023 < n_2023; i_2023++)
            System.out.print(arr_2023[i_2023] + " ");
        System.out.println("");
        insertionSort_2023(arr_2023);
        System.out.printf("Array yang terurut:\n");
        for (int i_2023 = 0; i_2023 < n_2023; i_2023++)
            System.out.print(arr_2023[i_2023] + " ");
        System.out.println("");
    }
}
```

Penjelasan Kode:

- `int minIndex_2023 = i_2023;`: Program mengasumsikan bahwa elemen pada indeks `i` saat ini adalah nilai terkecil (minimum).
- `for (int j_2023 = i_2023 + 1; ...)`: Perulangan dalam (inner loop) bertugas menelusuri sisa array di sebelah kanan untuk memverifikasi apakah ada angka yang lebih kecil dari asumsi awal.
- `if (arr_2023[j_2023] < arr_2023[minIndex_2023])`: Jika ditemukan angka yang lebih kecil, label `minIndex` akan dipindahkan ke indeks `j` tersebut.

- Proses Swap (variabel temp): Setelah satu putaran selesai, angka terkecil yang ditemukan akan ditukar secara fisik posisinya dengan indeks i.

Output:



```

Console X
<terminated> InsertionSort_251153202
Array yang belum terurut:
23 78 45 8 32 56 1
Array yang terurut:
1 8 23 32 45 56 78

```

### 2.3.2 Kelas Selection Sort

Kelas ini menggunakan teknik penyisipan data pada blok indeks yang sudah terurut sebelumnya.

```

package pekan7_2511532023;

public class SelectionSort_2511532023 {
    public static void selectionSort_2023(int[] arr_2023) {
        int n_2023 = arr_2023.length;
        for (int i_2023 = 0; i_2023 < n_2023; i_2023++) {
            int minIndex_2023 = i_2023;
            for (int j_2023 = i_2023 + 1; j_2023 < n_2023;
j_2023++) {
                if (arr_2023[j_2023] <
arr_2023[minIndex_2023]) {
                    minIndex_2023 = j_2023;    }
            }
            int temp_2023 = arr_2023[i_2023];
            arr_2023[i_2023] = arr_2023[minIndex_2023];
            arr_2023[minIndex_2023] = temp_2023;    }
        }
    public static void main(String[] args) {
        int arr_2023[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_2023 = arr_2023.length;
        System.out.printf("Array yang belum terurut:\n");
        for (int i_2023 = 0; i_2023 < n_2023; i_2023++)
            System.out.print(arr_2023[i_2023] + " ");
        System.out.println("");
        selectionSort_2023(arr_2023);
        System.out.printf("Array yang terurut:\n");
        for (int i_2023 = 0; i_2023 < n_2023; i_2023++)
            System.out.print(arr_2023[i_2023] + " ");
        System.out.println("");    }
}

```

Penjelasan Kode:



- `int i_2023 = 1`: Iterasi dimulai dari indeks ke-1, bukan ke-0. Elemen ke-0 dianggap sebagai sub-array yang sudah terurut.
- `int key_2023 = arr_2023[i_2023]`:: Nilai yang sedang dievaluasi disimpan ke dalam variabel `key` sebagai "kartu" yang akan disisipkan.
- `while (j_2023 >= 0 && arr_2023[j_2023] > key_2023)`: Selama angka di sebelah kiri `key` lebih besar, maka angka-angka besar tersebut digeser satu langkah ke kanan (`arr_2023[j_2023 + 1] = arr_2023[j_2023]`).
- Setelah posisi yang tepat ditemukan (ketika perulangan `while` berhenti), `key` dimasukkan ke celah yang kosong.

Output:

```
<terminated> SelectionSort_2511532023 [
Array yang belum terurut:
23 78 45 8 32 56 1
Array yang terurut:
1 8 23 32 45 56 78
```

### 2.3.3 Kelas Bubble Sort

Algoritma ini merupakan metode paling sederhana namun membutuhkan komputasi perbandingan yang cukup banyak di setiap putarannya.

```
package pekan7_2511532023;

public class BubbleSort_2511532023 {
    public static void bubbleSort_2023(int[] arr_2023) {
        int n_2023 = arr_2023.length;
        for (int i_2023 = 0; i_2023 < n_2023; i_2023++) {
            for (int j_2023 = 0; j_2023 < n_2023 - i_2023 - 1; j_2023++) {
                if (arr_2023[j_2023] > arr_2023[j_2023 + 1])
                {
                    int temp_2023 = arr_2023[j_2023];
                    arr_2023[j_2023] = arr_2023[j_2023 + 1];
                    arr_2023[j_2023 + 1] = temp_2023;
                    // System.out.println("Data: " +
                    arr_2023[j_2023] + " " + arr_2023[j_2023 + 1]);
                }
            }
        }
    }
}
```

```

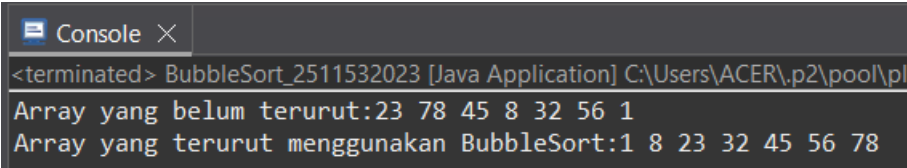
    }
    public static void main(String[] args) {
        int arr_2023[] = { 23, 78, 45, 8, 32, 56, 1 };
        int n_2023 = arr_2023.length;
        System.out.printf("Array yang belum terurut:");
        for (int i_2023 = 0; i_2023 < n_2023; i_2023++)
            System.out.print(arr_2023[i_2023] + " ");
        System.out.println("");
        bubbleSort_2023(arr_2023);
        System.out.printf("Array yang terurut menggunakan
BubbleSort:");
        for (int i_2023 = 0; i_2023 < n_2023; i_2023++)
            System.out.print(arr_2023[i_2023] + " ");
        System.out.println("");
    }
}

```

Penjelasan Kode:

- for (int j\_2023 = 0; j\_2023 < n\_2023 - i\_2023 - 1; j\_2023++):  
Perulangan membandingkan dua indeks yang bersebelahan. Pengurangan - i\_2023 berfungsi mempercepat proses, karena elemen paling besar akan selalu "mengapung" ke indeks ujung kanan pada setiap akhir putaran luar i, sehingga tidak perlu dicek ulang.
- if (arr\_2023[j\_2023] > arr\_2023[j\_2023 + 1]): Merupakan kondisi mutlak pertukaran. Jika elemen sebelah kiri lebih besar dari elemen sebelah kanannya, posisi mereka langsung ditukar menggunakan variabel pembantu temp.

Output:



```

<terminated> BubbleSort_2511532023 [Java Application] C:\Users\ACER\p2\pool\pl
Array yang belum terurut:23 78 45 8 32 56 1
Array yang terurut menggunakan BubbleSort:1 8 23 32 45 56 78

```

### 2.3.4 Kelas Insertion Sort GUI

Kelas ini mengeksekusi animasi logika Sorting menggunakan library javax.swing dan java.awt. GUI dirancang interaktif untuk merekam log step-by-step setiap kali nilai digeser.

```

package pekan7_2511532023;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;

```

```

import java.awt.EventQueue;
import java.awt.Font;
import java.lang.reflect.Array;

import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JLabel;
import javax.swing.JFrame;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;
import javax.swing.border.EmptyBorder;

public class InsertionSortGUI_2511532023 extends JFrame {
    private static final long serialVersionUID = 1L;
    private int[] array_2023;
    private JLabel[] labelArray_2023;
    private JButton stepButton_2023, resetButton_2023,
setButton_2023;
    private JTextField inputField_2023;
    private JPanel panelArray_2023;
    private JTextArea stepArea_2023;

    private int i_2023 = 1, j_2023;
    private boolean sorting_2023 = false;
    private int stepCount_2023 = 1;

    /**
     * Create the frame.
     */
    public InsertionSortGUI_2511532023() {
        setTitle("Insertion Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_2023 = new JPanel(new FlowLayout());
        inputField_2023 = new JTextField(30);
        setButton_2023 = new JButton("Set Array");
        inputPanel_2023.add(new JLabel("Masukkan angka
(pisahkan dengan koma):"));
        inputPanel_2023.add(inputField_2023);
        inputPanel_2023.add(setButton_2023);

        // Panel array visual
        panelArray_2023 = new JPanel();
        panelArray_2023.setLayout(new FlowLayout());

```

```

// Panel kontrol
JPanel controlPanel = new JPanel();
stepButton_2023 = new JButton("Langkah Selanjutnya");
resetButton_2023 = new JButton("Reset");
stepButton_2023.setEnabled(false);
controlPanel.add(stepButton_2023);
controlPanel.add(resetButton_2023);

// Area teks untuk log langkah-langkah
stepArea_2023 = new JTextArea(8, 60);
stepArea_2023.setEditable(false);
stepArea_2023.setFont(new Font("Monospaced",
Font.PLAIN, 14));
JScrollPane scrollPane = new
JScrollPane(stepArea_2023);

// Tambahkan panel ke frame
add(inputPanel_2023, BorderLayout.NORTH);
add(panelArray_2023, BorderLayout.CENTER);
add(controlPanel, BorderLayout.SOUTH);
add(scrollPane, BorderLayout.EAST);

// Event Set Array
setButton_2023.addActionListener(e ->
setArrayFromInput_2023());

// Event Langkah Selanjutnya
stepButton_2023.addActionListener(e ->
performStep_2023());

// Event Reset
resetButton_2023.addActionListener(e -> reset_2023());
}

private void setArrayFromInput_2023() {
String text_2023 =
inputField_2023.getText().trim();
if (text_2023.isEmpty()) return;
String[] parts = text_2023.split(",");
array_2023 = new int[parts.length];
try {
for (int k_2023 = 0; k_2023 <
parts.length; k_2023++) {
array_2023[k_2023] =
Integer.parseInt(parts[k_2023].trim()); }
} catch (NumberFormatException e) {
JOptionPane.showMessageDialog(this,
"Masukkan hanya angka yang dipisahkan " + "dengan koma!", "Error",
JOptionPane.ERROR_MESSAGE);
return; }
i_2023 = 1;
stepCount_2023 = 1;
sorting_2023 = true;
stepButton_2023.setEnabled(true);
stepArea_2023.setText("");
}

```

```

        panelArray_2023.removeAll();
        labelArray_2023 = new
JLabel[array_2023.length];
        for (int k_2023 = 0; k_2023 <
array_2023.length; k_2023++) {
            labelArray_2023[k_2023] = new
JLabel(String.valueOf(array_2023[k_2023]));
            labelArray_2023[k_2023].setFont(new
Font("Arial", Font.BOLD, 24));

            labelArray_2023[k_2023].setBorder(BorderFactory.createLineBorde
r(Color.BLACK));

            labelArray_2023[k_2023].setPreferredSize(new Dimension(50,
50));

            labelArray_2023[k_2023].setHorizontalAlignment(SwingConstants.C
ENTER);

            panelArray_2023.add(labelArray_2023[k_2023]);
        }
        panelArray_2023.revalidate();
        panelArray_2023.repaint(); }

private void performStep_2023() {
    if (i_2023 < array_2023.length && sorting_2023) {
        int key_2023 = array_2023[i_2023];
        j_2023 = i_2023 - 1;

        StringBuilder stepLog_2023 = new
StringBuilder();
        stepLog_2023.append("Langkah
").append(stepCount_2023).append(": Memasukkan
").append(key_2023).append("\n");

        while (j_2023 >= 0 && array_2023[j_2023] >
key_2023) {
            array_2023[j_2023 + 1] =
array_2023[j_2023];
            j_2023--;
        }
        array_2023[j_2023 + 1] = key_2023;

        updateLabels_2023();
        stepLog_2023.append("Hasil:
").append(arrayToString_2023(array_2023)).append("\n\n");
        stepArea_2023.append(stepLog_2023.toString());

        i_2023++;
        stepCount_2023++;

        if (i_2023 == array_2023.length) {
            sorting_2023 = false;
            stepButton_2023.setEnabled(false);

```

```

JOptionPane.showMessageDialog(this,
"Sorting selesai!");
    }
}
private void updateLabels_2023() {
    for (int k_2023 = 0; k_2023 < array_2023.length;
k_2023++) {
labelArray_2023[k_2023].setText(String.valueOf(array_2023[k_2023]));
    }
}
private void reset_2023() {
    inputField_2023.setText("");
    panelArray_2023.removeAll();
    panelArray_2023.revalidate();
    panelArray_2023.repaint();
    stepArea_2023.setText("");
    stepButton_2023.setEnabled(false);
    sorting_2023 = false;
    i_2023 = 1;
    stepCount_2023 = 1;
}

private String arrayToString_2023(int[] arr_2023) {
    StringBuilder sb_2023 = new StringBuilder();
    for (int k_2023 = 0; k_2023 < arr_2023.length;
k_2023++) {
        sb_2023.append(arr_2023[k_2023]);
        if (k_2023 < arr_2023.length - 1)
sb_2023.append(", ");
    }
    return sb_2023.toString();
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(() -> {
        InsertionSortGUI_2511532023 gui_2023 = new
InsertionSortGUI_2511532023();
        gui_2023.setVisible(true);
    });
}
}

```

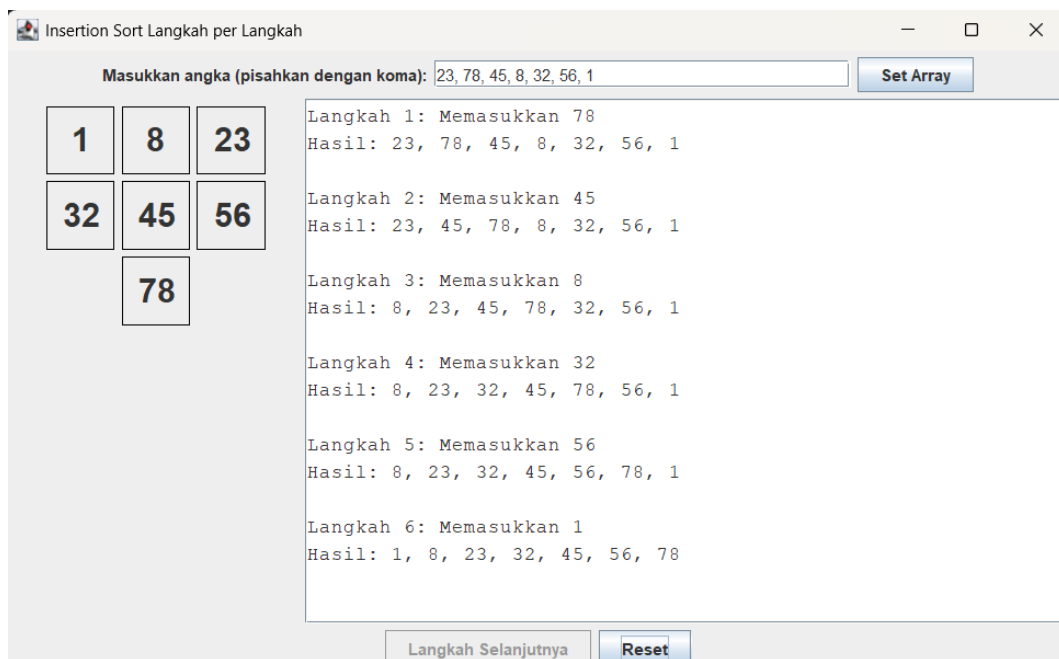
#### Penjelasan Kode:

- extends JFrame: Mendeklarasikan bahwa kelas utama merupakan pewarisan (inheritance) dari komponen bingkai jendela utama di Java Swing.
- Mekanisme performStep\_2023() adalah jantung pemrosesan. Tidak seperti eksekusi konsol biasa yang memproses perulangan for secara instan, GUI

ini memanfaatkan penekanan tombol (Action Event) untuk men-trigger iterasi satu per satu, mendemonstrasikan perubahan key dan pergeseran indeks di layar JLabel.

- StringBuilder digunakan untuk merangkai string teks riwayat perpindahan array agar efisien memori sebelum ditampilkan secara akumulatif ke komponen JTextArea.

Output:



## BAB III

### PENUTUP

#### 3.1 Kesimpulan

Berdasarkan hasil praktikum dan pembahasan kode program, dapat disimpulkan bahwa algoritma *Sorting* memegang peran yang sangat penting dalam menstrukturisasi memori data acak. Bubble Sort bekerja optimal untuk data yang jumlahnya sangat kecil karena sifat kodenya yang intuitif. Selection Sort meminimalkan jumlah *swap* (pertukaran) memori dibandingkan Bubble Sort, namun tetap melakukan komputasi perbandingan yang kaku. Di sisi lain, Insertion Sort terbukti sangat efisien jika sekumpulan data sudah terurut secara parsial (mendekati Best Case  $O(n)$ ). Implementasi menggunakan Java Swing juga membuktikan bahwa logika algoritma struktural sangat relevan disinergikan dengan *Event-Driven Programming* untuk menciptakan aplikasi berbasis antarmuka visual.

#### 3.2 Saran

Dalam merancang logika pengurutan kompleks maupun memvisualisasikannya di antarmuka grafis ke depannya, sangat disarankan untuk berhati-hati dalam mendefinisikan batasan indeks pada array (contoh:  $n - i - 1$ ) untuk menghindari pesan kesalahan mutlak *ArrayIndexOutOfBoundsException*. Mahasiswa juga disarankan untuk mengeksplorasi algoritma rekursif yang jauh lebih efisien untuk skala besar seperti Merge Sort maupun Quick Sort, serta menerapkan pendekatan *Model-View-Controller* (MVC) jika kode GUI mulai terlalu panjang dan tercampur dengan logika komputasi murni.



## DAFTAR PUSTAKA

- [1] D4 Manajemen Informatika, "Algoritma Sort: Pengertian, Jenis, Cara Kerja, dan Contoh Lengkap," Universitas Negeri Surabaya. [Daring]. Tersedia pada: <https://terapan-ti.vokasi.unesa.ac.id/post/algoritma-sort-pengertian-jenis-cara-kerja-dan-contoh-lengkap>. [Diakses: 20-Mei-2026].
- [2] Universitas Siliwangi, "Laporan Praktikum Dasar Pemrograman - Sorting," Studocu. [Daring]. Tersedia pada: <https://www.studocu.id/id/document/universitas-siliwangi/praktikum-dasar-pemrograman/laporan-praktikum-sorting/41638317>. [Diakses: 21-Mei-2026].
- [3] Politeknik Elektronika Negeri Surabaya, "Praktikum 9-10 - Sorting Insertion dan Selection," PENS. [Daring]. Tersedia pada: [https://arna.lecturer.pens.ac.id/PrakASD\\_Java/Praktikum%209-10%20-%20Sorting%20Insertion%20dan%20Selection.pdf](https://arna.lecturer.pens.ac.id/PrakASD_Java/Praktikum%209-10%20-%20Sorting%20Insertion%20dan%20Selection.pdf). [Diakses: 21-Mei-2026].
- [4] BINUS University Bandung, "Implementasi Algoritma Bubble Sort dengan Bahasa Pemrograman Python," Okt. 2023. [Daring]. Tersedia pada: <https://binus.ac.id/bandung/2023/10/implementasi-algoritma-bubble-sort-dengan-bahasa-pemrograman-python/>. [Diakses: 20-Mei-2026].

## TUGAS 7 PRAKTIKUM STRUKTUR DATA

### 1. Soal

Meminta pengembangan sebuah aplikasi antarmuka (GUI) menggunakan Java Swing untuk menyimulasikan dan memvisualisasikan proses pengurutan data mahasiswa secara interaktif. Aplikasi ini dirancang untuk membandingkan tiga algoritma sorting dasar dalam mengurutkan data teks (*String*) secara *ascending* (A-Z).

### 2. Jawab

- a) Kelas ADT Mahasiswa

```
package pekan7_2511532023;

public class Mahasiswa_2511532023 {
    // Atribut
    private String nama_2023;
    private String nim_2023;
    private String prodi_2023;

    // Konstruktor
    public Mahasiswa_2511532023(String nama_2023, String nim_2023,
String prodi_2023) {
        this.nama_2023 = nama_2023;
        this.nim_2023 = nim_2023;
        this.prodi_2023 = prodi_2023;
    }

    // Getter
    public String getNama_2023() {
        return nama_2023;
    }
    public String getNim_2023() {
        return nim_2023;
    }
    public String getProdi_2023() {
        return prodi_2023;
    }

    // Setter
    public void setNama_2023(String nama_2023) {
        this.nama_2023 = nama_2023;
    }
    public void setNim_2023(String nim_2023) {
        this.nim_2023 = nim_2023;
    }
    public void setProdi(String prodi_2023) {
        this.prodi_2023 = prodi_2023;
    }

    // toString()
    @Override
```

```

    public String toString() {
        return nama_2023;
    }
}

```

Penjelasan Kode:

- private String nama\_2023 / nim\_2023 / prodi\_2023;: Atribut rahasia (private) untuk menyimpan data nama, nim, dan prodi mahasiswa
- public Mahasiswa\_2511532023(...): konstruktor untuk memasukkan data input awal langsung ke atribut objek menggunakan kata kunci this.
- getNama\_2023() / getNim\_2023() / getProdi\_2023(): Method Getter untuk mengizinkan kelas lain mengambil nilai dari masing-masing atribut mahasiswa.
- setNama\_2023() / setNim\_2023() / setProdi\_2023(): Method Setter untuk mengubah atau mengisi ulang nilai masing-masing atribut mahasiswa dari luar kelas.
- @Override public String toString(): Mengubah fungsi cetak bawaan Java agar saat list dipanggil, sistem otomatis hanya mengeluarkan teks nama mahasiswanya saja.

b) Kelas Method Sorting

```

package pekan7_2511532023;

import java.util.ArrayList;

public class Sorting_2511532023 {

    // Insertion Sort
    public static String
    insertionSort_2023(ArrayList<Mahasiswa_2511532023> list) {
        StringBuilder log_2023 = new StringBuilder("=== INSERTION
SORT ===\n");
        int n_2023 = list.size();

        for (int i_2023 = 1; i_2023 < n_2023; i_2023++) {
            Mahasiswa_2511532023 key_2023 = list.get(i_2023); //
Data yang mau disisipkan
            int j_2023 = i_2023 - 1;

            // Geser data yang lebih besar ke kanan untuk
memberi ruang bagi 'key'
            while (j_2023 >= 0 &&
list.get(j_2023).getNama_2023().compareToIgnoreCase(key_2023.getNama_2023()) > 0) {
                list.set(j_2023 + 1, list.get(j_2023));
            }
        }
    }
}

```

```

        j_2023--;
    }
    // Sisipkan key
    list.set(j_2023 + 1, key_2023);

    // Simpan langkah ke String untuk GUI dan cetak ke
console
    String langkah_2023 = "Langkah " + i_2023 + ": " +
list.toString();
    log_2023.append(langkah_2023).append("\n");
    System.out.println(langkah_2023);
}
return log_2023.toString();
}

// Selection Sort
public static String
selectionSort_2023(ArrayList<Mahasiswa_2511532023> list) {
    StringBuilder log_2023 = new StringBuilder("=== SELECTION
SORT ===\n");
    int n_2023 = list.size();

    for (int i_2023 = 0; i_2023 < n_2023 - 1; i_2023++) {
        int indexMin = i_2023; // Misal posisi i adalah yang
terkecil sementara

        for(int j_2023 = i_2023 + 1; j_2023 < n_2023;
j_2023++) {
            if
(list.get(j_2023).getNama_2023().compareToIgnoreCase(list.get(indexMin).
getNama_2023()) < 0) {
                indexMin = j_2023;
            }
        }
        // Tukar data di posisi i dengan data terkecil yang
baru ditemukan.
        Mahasiswa_2511532023 temp_2023 = list.get(indexMin);
        list.set(indexMin, list.get(i_2023));
        list.set(i_2023, temp_2023);

        // Simpan langkah ke String untuk GUI dan cetak ke
console
        String langkah_2023 = "Pass " + (i_2023 + 1) + ": " +
list.toString();
        log_2023.append(langkah_2023).append("\n");
        System.out.println(langkah_2023);
    }
    return log_2023.toString();
}

// Bubble Sort
public static String
bubbleSort_2023(ArrayList<Mahasiswa_2511532023> list) {
    StringBuilder log_2023 = new StringBuilder("=== BUBBLE SORT
===\n");
    int n_2023 = list.size();

```

```

        for(int i_2023 = 0; i_2023 < n_2023 - 1; i_2023++) {
            for(int j_2023 = 0; j_2023 < n_2023 - i_2023 - 1;
j_2023++) {
                // Ambil nama dari mahasiswa posisi j dan
j+1, lalu bandingkan
                if(list.get(j_2023).getNama_2023().compareToIgnoreCase(list.get(j
_2023 + 1).getNama_2023()) > 0) {
                    // Tukar posisi jika nama kiri lebih
besar dari nama kanan
                    Mahasiswa_2511532023 temp_2023 =
list.get(j_2023);
                    list.set(j_2023, list.get(j_2023 + 1));
                    list.set(j_2023 + 1, temp_2023);
                }
            }
            // Simpan langkah ke String untuk GUI dan cetak ke
console
            String langkah_2023 = "Pass " + (i_2023 + 1) + ": "
+ list.toString();
            log_2023.append(langkah_2023).append("\n");
            System.out.println(langkah_2023);
        }
        return log_2023.toString();
    }
}

```

#### Penjelasan Kode:

- public static String insertionSort\_2023 / selectionSort\_2023 / bubbleSort\_2023 (...): Kumpulan method statis algoritma pengurutan yang menerima daftar data awal (ArrayList), mengurutkannya, dan mengembalikan string rekaman proses.
- StringBuilder log\_2023: Wadah teks untuk menyambung catatan riwayat langkah pengurutan secara cepat dan efisien.
- int n\_2023 = list.size();: Mengambil total jumlah mahasiswa di dalam daftar untuk menentukan batas akhir perulangan.
- for (int i\_2023 ... ) & for (int j\_2023 ... ): Kombinasi *Outer Loop* untuk mengatur jumlah putaran dan *Inner Loop* untuk memindai posisi index data.
- compareToIgnoreCase(...): Method untuk membandingkan urutan abjad nama (A-Z) secara akurat tanpa memedulikan huruf kapital atau kecil.

- Mahasiswa\_2511532023 temp\_2023 = ... / list.set(...): Logika penukaran posisi (*Swap*) untuk menggeser objek mahasiswa yang salah urutan di dalam memori komputer.
- log\_2023.append(...) & System.out.println(...): Merekam susunan nama terkini di setiap putaran untuk ditampilkan ke area log GUI sekaligus mencetaknya ke console bawah Eclipse.

c) Kelas GUI

```
package pekan7_2511532023;

import javax.swing.*;
import javax.swing.table.DefaultTableModel;
import java.awt.*;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.ArrayList;

public class MahasiswaSortingGUI_2511532023 extends JFrame {

    private ArrayList<Mahasiswa_2511532023> listAsli_2023 = new
ArrayList<>();

    private JTextField txtNama_2023, txtNim_2023, txtProdi_2023;
    private JButton btnTambah_2023, btnHapus_2023, btnSort_2023;
    private JComboBox<String> cmbAlgo_2023;
    private JTable tabelData_2023;
    private DefaultTableModel modelTabel_2023;
    private JTextArea txtAreaLog_2023;

    public MahasiswaSortingGUI_2511532023() {
        setTitle("Pengurutan Mahasiswa - 2511532023");
        setSize(500, 650);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout(8, 8));

        JPanel panelAtas_2023 = new JPanel(new BorderLayout(5, 5));
        JPanel panelForm_2023 = new JPanel(new GridLayout(3, 2, 5, 5));
        panelForm_2023.setBorder(BorderFactory.createTitledBorder("Form
Input Mahasiswa"));

        panelForm_2023.add(new JLabel(" Nama Mahasiswa: "));
        txtNama_2023 = new JTextField();
        panelForm_2023.add(txtNama_2023);

        panelForm_2023.add(new JLabel(" NIM Mahasiswa: "));
        txtNim_2023 = new JTextField();
        panelForm_2023.add(txtNim_2023);

        panelForm_2023.add(new JLabel(" Program Studi: "));
        txtProdi_2023 = new JTextField();
        panelForm_2023.add(txtProdi_2023);
    }
}
```

```

JPanel panelTombolData_2023 = new JPanel(new GridLayout(1, 2, 5,
5));
btnTambah_2023 = new JButton("Tambah Data");
btnHapus_2023 = new JButton("Hapus Terpilih");
panelTombolData_2023.add(btnTambah_2023);
panelTombolData_2023.add(btnHapus_2023);

panelAtas_2023.add(panelForm_2023, BorderLayout.CENTER);
panelAtas_2023.add(panelTombolData_2023, BorderLayout.SOUTH);

JPanel panelTengah_2023 = new JPanel(new GridLayout(2, 1, 5,
5));

String[] kolom_2023 = {"Nama", "NIM", "Prodi"};
modelTabel_2023 = new DefaultTableModel(kolom_2023, 0);
tabelData_2023 = new JTable(modelTabel_2023);
JScrollPane scrollTabel_2023 = new JScrollPane(tabelData_2023);

scrollTabel_2023.setBorder(BorderFactory.createTitledBorder("Daftar
Mahasiswa (Data Awal)"));
panelTengah_2023.add(scrollTabel_2023);

txtAreaLog_2023 = new JTextArea();
txtAreaLog_2023.setEditable(false);
txtAreaLog_2023.setFont(new Font("Monospaced", Font.PLAIN, 12));
JScrollPane scrollLog_2023 = new JScrollPane(txtAreaLog_2023);

scrollLog_2023.setBorder(BorderFactory.createTitledBorder("Visualisasi
Langkah Sorting"));
panelTengah_2023.add(scrollLog_2023);

JPanel panelKontrol_2023 = new JPanel(new
FlowLayout(FlowLayout.CENTER, 10, 5));

panelKontrol_2023.setBorder(BorderFactory.createTitledBorder("Pilihan
Urutan"));

panelKontrol_2023.add(new JLabel("Algoritma:"));
String[] pilihanAlgo_2023 = {"Insertion Sort", "Selection Sort",
"Bubble Sort"};
cmbAlgo_2023 = new JComboBox<>(pilihanAlgo_2023);
panelKontrol_2023.add(cmbAlgo_2023);

btnSort_2023 = new JButton("Mulai Sorting");
panelKontrol_2023.add(btnSort_2023);

JPanel panelKontainerUtama_2023 = new JPanel(new BorderLayout(5,
5));

panelKontainerUtama_2023.setBorder(BorderFactory.createEmptyBorder(10,
10, 10, 10));
panelKontainerUtama_2023.add(panelAtas_2023,
BorderLayout.NORTH);
panelKontainerUtama_2023.add(panelTengah_2023,
BorderLayout.CENTER);

```

```

        panelKontainerUtama_2023.add(panelKontrol_2023,
BorderLayout.SOUTH);

        add(panelKontainerUtama_2023);

        btnTambah_2023.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                String nama_2023 = txtNama_2023.getText().trim();
                String nim_2023 = txtNim_2023.getText().trim();
                String prodi_2023 = txtProdi_2023.getText().trim();

                if (!nama_2023.isEmpty() && !nim_2023.isEmpty() &&
!prodi_2023.isEmpty()) {
                    Mahasiswa_2511532023 mhs_2023 = new
Mahasiswa_2511532023(nama_2023, nim_2023, prodi_2023);
                    listAsli_2023.add(mhs_2023);
                    modelTabel_2023.addRow(new Object[]{nama_2023,
nim_2023, prodi_2023});

                    txtNama_2023.setText("");
                    txtNim_2023.setText("");
                    txtProdi_2023.setText("");
                } else {
                    JOptionPane.showMessageDialog(null, "Semua inputan
wajib diisi!");
                }
            }
        });

        btnHapus_2023.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                int barisTerpilih_2023 =
tabelData_2023.getSelectedRow();
                if (barisTerpilih_2023 != -1) {
                    listAsli_2023.remove(barisTerpilih_2023);
                    modelTabel_2023.removeRow(barisTerpilih_2023);
                } else {
                    JOptionPane.showMessageDialog(null, "Silakan pilih
baris tabel yang ingin dihapus!");
                }
            }
        });

        btnSort_2023.addActionListener(new ActionListener() {
            @Override
            public void actionPerformed(ActionEvent e) {
                // Cek apakah data mhs kosong sebelum diurutkan
                if (listAsli_2023.isEmpty()) {
                    JOptionPane.showMessageDialog(null, "Data mahasiswa
masih kosong!");
                    return;
                }

                // Clear log sebelumnya
                txtAreaLog_2023.setText("");
            }
        });

```



```

berubah // Duplikasi data awal agar urutan data pada tabel tidak
berubah
ArrayList<Mahasiswa_2511532023> listDuplikat_2023 = new
ArrayList<>();
for (Mahasiswa_2511532023 mhs_2023 : listAsli_2023) {
    listDuplikat_2023.add(new Mahasiswa_2511532023(
        mhs_2023.getNama_2023(),
        mhs_2023.getNim_2023(),
        mhs_2023.getProdi_2023()
    ));
}

// Deteksi pilihan algo dari CmbBox dan eksekusi sorting
String pilihan_2023 = (String)
cmbAlgo_2023.getSelectedItem();
String hasilLog_2023 = "";

if (pilihan_2023.equals("Insertion Sort")) {
    hasilLog_2023 =
Sorting_2511532023.insertionSort_2023(listDuplikat_2023);
} else if (pilihan_2023.equals("Selection Sort")) {
    hasilLog_2023 =
Sorting_2511532023.selectionSort_2023(listDuplikat_2023);
} else if (pilihan_2023.equals("Bubble Sort")) {
    hasilLog_2023 =
Sorting_2511532023.bubbleSort_2023(listDuplikat_2023);
}

// Tampilkan riwayat langkah visualisasi ke JTextArea
txtAreaLog_2023.setText(hasilLog_2023);
}
});
}

public static void main(String[] args) {
    SwingUtilities.invokeLater(new Runnable() {
        @Override
        public void run() {
            new MahasiswaSortingGUI_2511532023().setVisible(true);
        }
    });
}
}

```

#### Penjelasan Kode:

- private ArrayList<Mahasiswa\_2511532023> listAsli\_2023: Tempat penyimpanan pusat untuk menjaga data awal mahasiswa tetap utuh dan tidak rusak saat diacak oleh mesin sorting.

- txtNama\_2023 / txtNim\_2023 / txtProdi\_2023 / txtAreaLog\_2023: Komponen visual untuk mengetik data input mahasiswa dan area teks besar untuk menampilkan hasil visualisasi sorting.
- btnTambah\_2023 / btnHapus\_2023 / btnSort\_2023 / cmbAlgo\_2023: Komponen tombol aksi kontrol data (tambah/hapus), tombol eksekusi, serta menu tarik-turun (dropdown) pilihan algoritma.
- public MahasiswaSortingGUI\_2511532023(): Konstruktor GUI untuk merancang ukuran jendela dan menyusun tata letak panel komponen agar presisi.
- btnTambah\_2023 / btnHapus\_2023.addActionListener(...): Sensor tombol kelola data, memasukkan data form teks ke dalam tabel dan list asli, atau menghapus baris tabel yang dipilih pengguna.
- for (Mahasiswa\_2511532023 mhs\_2023 : listAsli\_2023): Logika Deep Copy: Menyalin isi objek dari list asli ke list duplikat baru secara mandiri agar pengujian ketiga algoritma bersifat adil dengan data awal yang identik.
- if (pilihan\_2023.equals(...)) & txtAreaLog\_2023.setText(...): Membaca pilihan dropdown, memanggil method pengurutan yang cocok dari kelas Sorting\_2511532023, lalu menempelkan string hasilnya ke layar teks GUI.
- public static void main(String[] args) Penghubung utama sistem untuk menghidupkan dan menampilkan jendela aplikasi ke layar monitor.

Output:

- Insertion Sort

Pengurutan Mahasiswa - 2511532023

**Form Input Mahasiswa**

Nama Mahasiswa:

NIM Mahasiswa:

Program Studi:

**Tambah Data** **Hapus Terpilih**

**Daftar Mahasiswa (Data Awal)**

| Nama  | NIM  | Prodi |
|-------|------|-------|
| Maya  | 2025 | SI    |
| Rizki | 1005 | TK    |
| Caca  | 2005 | IF    |
| Andi  | 3001 | IF    |

**Visualisasi Langkah Sorting**

```

=== INSERTION SORT ===
Langkah 1: [Maya, Rizki, Caca, Andi]
Langkah 2: [Caca, Maya, Rizki, Andi]
Langkah 3: [Andi, Caca, Maya, Rizki]

```

**Pilihan Urutan**

Algoritma: **Insertion Sort** **Mulai Sorting**

- Selection Sort

Pengurutan Mahasiswa - 2511532023

---

**Form Input Mahasiswa**

Nama Mahasiswa:

NIM Mahasiswa:

Program Studi:

**Tambah Data** **Hapus Terpilih**

**Daftar Mahasiswa (Data Awal)**

| Nama  | NIM  | Prodi |
|-------|------|-------|
| Maya  | 2025 | SI    |
| Rizki | 1005 | TK    |
| Caca  | 2005 | IF    |
| Andi  | 3001 | IF    |

**Visualisasi Langkah Sorting**

```

=== SELECTION SORT ===
Pass 1: [Andi, Rizki, Caca, Maya]
Pass 2: [Andi, Caca, Rizki, Maya]
Pass 3: [Andi, Caca, Maya, Rizki]

```

**Pilihan Urutan**

Algoritma: **Selection Sort** **Mulai Sorting**

- Bubble Sort

Pengurutan Mahasiswa - 2511532023

**Form Input Mahasiswa**

Nama Mahasiswa:

NIM Mahasiswa:

Program Studi:

**Tambah Data** **Hapus Terpilih**

**Daftar Mahasiswa (Data Awal)**

| Nama  | NIM  | Prodi |
|-------|------|-------|
| Maya  | 2025 | SI    |
| Rizki | 1005 | TK    |
| Caca  | 2005 | IF    |
| Andi  | 3001 | IF    |

**Visualisasi Langkah Sorting**

```
=== BUBBLE SORT ===  
Pass 1: [Maya, Caca, Andi, Rizki]  
Pass 2: [Caca, Andi, Maya, Rizki]  
Pass 3: [Andi, Caca, Maya, Rizki]
```

**Pilihan Urutan**

Algoritma: **Bubble Sort** **Mulai Sorting**